

# VyaparFlow

## Low-Level Design (LLD)

Folder Structure, Modules, Classes, Database Schema, and Implementation Contracts

*This LLD translates the HLD into an implementation blueprint. It defines the recommended backend folder structure, module boundaries, classes, interfaces, database schema, DTOs, validation rules, and service responsibilities for the MVP.*

|                   |                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| Scope             | MVP implementation structure for voice commands, agents, inventory, finance, analytics, and notifications |
| Suggested Backend | Node.js + NestJS + TypeScript, with optional Python AI worker service                                     |
| Database          | PostgreSQL as source of truth, Redis for cache/workflow state                                             |
| Design Style      | Modular monolith for MVP, cleanly separable into services later                                           |
| Core Pattern      | Controller -> Service -> Repository, Agent tools call domain services                                     |
| Important Rule    | Agents plan workflows; domain services validate and write data                                            |

## 1. LLD Opening

### Presenter:

In this Low-Level Design, I will explain how VyaparFlow can be implemented at the code, module, class, and database level. The HLD described the architecture. This LLD goes one level deeper and defines the folder structure, core modules, classes, database tables, DTOs, validation rules, and internal contracts.

### Presenter:

For the MVP, the recommended approach is a modular monolith backend. This means all core modules live in one deployable backend application, but each module has clear boundaries. Later, modules like Agent, Speech, Analytics, or Notification can be extracted into independent services without redesigning the full system.

## 2. Recommended Repository Structure

```
vyaparflow/  
  apps/  
    web/                # React/Next.js frontend  
    api/                # NestJS backend API  
    ai-worker/          # Optional Python worker for AI-heavy jobs  
  packages/  
    shared-types/       # Shared DTOs and enums  
    config/             # Shared environment schema/config  
  infra/  
    docker/  
    migrations/  
    seed/  
  docs/  
    srs/  
    hld/  
    lld/  
  tests/  
    e2e/  
    fixtures/
```

The repository is separated by application and shared packages. The web app, backend API, and optional AI worker can evolve independently while sharing DTOs and environment definitions.

## 3. Backend Folder Structure

```
apps/api/src/  
  main.ts  
  app.module.ts  
  config/  
    env.schema.ts  
    database.config.ts  
    redis.config.ts  
  common/  
    decorators/  
    filters/  
    guards/  
    interceptors/  
    pipes/  
    utils/  
  modules/  
    auth/  
    users/  
    businesses/  
    voice/  
    nlp/  
    agent/  
    inventory/  
    finance/  
    analytics/  
    customers/  
    invoices/  
    notifications/  
    audit/  
  database/  
    prisma.schema or typeorm.entities/
```

```
migrations/  
jobs/  
  queue.module.ts  
workers/  
integrations/  
  speech-provider/  
  llm-provider/  
  storage-provider/  
  payment-provider/
```

## 4. Module Responsibility Matrix

| Module        | Responsibility                                                | Main Classes                                               |
|---------------|---------------------------------------------------------------|------------------------------------------------------------|
| Auth          | Login, token validation, role checks, tenant context.         | AuthController, AuthService, JwtStrategy, RolesGuard       |
| Users         | User profile, language preference, role assignment.           | UsersController, UsersService, UserRepository              |
| Businesses    | Business profile, settings, owner relationship.               | BusinessController, BusinessService, BusinessRepository    |
| Voice         | Audio upload, transcript creation, response formatting.       | VoiceController, VoiceCommandService, AudioStorageService  |
| NLP           | Language detection, intent classification, entity extraction. | NlpService, IntentClassifier, EntityExtractor              |
| Agent         | Workflow planning, tool selection, task execution tracking.   | AgentController, SupervisorAgentService, AgentToolRegistry |
| Inventory     | Products, stock movements, low-stock checks.                  | ProductService, InventoryService, StockMovementRepository  |
| Finance       | Sales, expenses, ledgers, profit calculations.                | TransactionService, ExpenseService, LedgerService          |
| Analytics     | Dashboard summaries, insights, trend calculations.            | AnalyticsService, InsightService, ReportService            |
| Customers     | Customer profile, dues, purchase history.                     | CustomerService, CustomerRepository                        |
| Invoices      | Invoice generation and invoice file storage.                  | InvoiceService, PdfInvoiceGenerator                        |
| Notifications | Reminders, low-stock alerts, future WhatsApp hooks.           | NotificationService, ReminderService                       |
| Audit         | Immutable record of user and agent actions.                   | AuditLogService, AuditRepository                           |

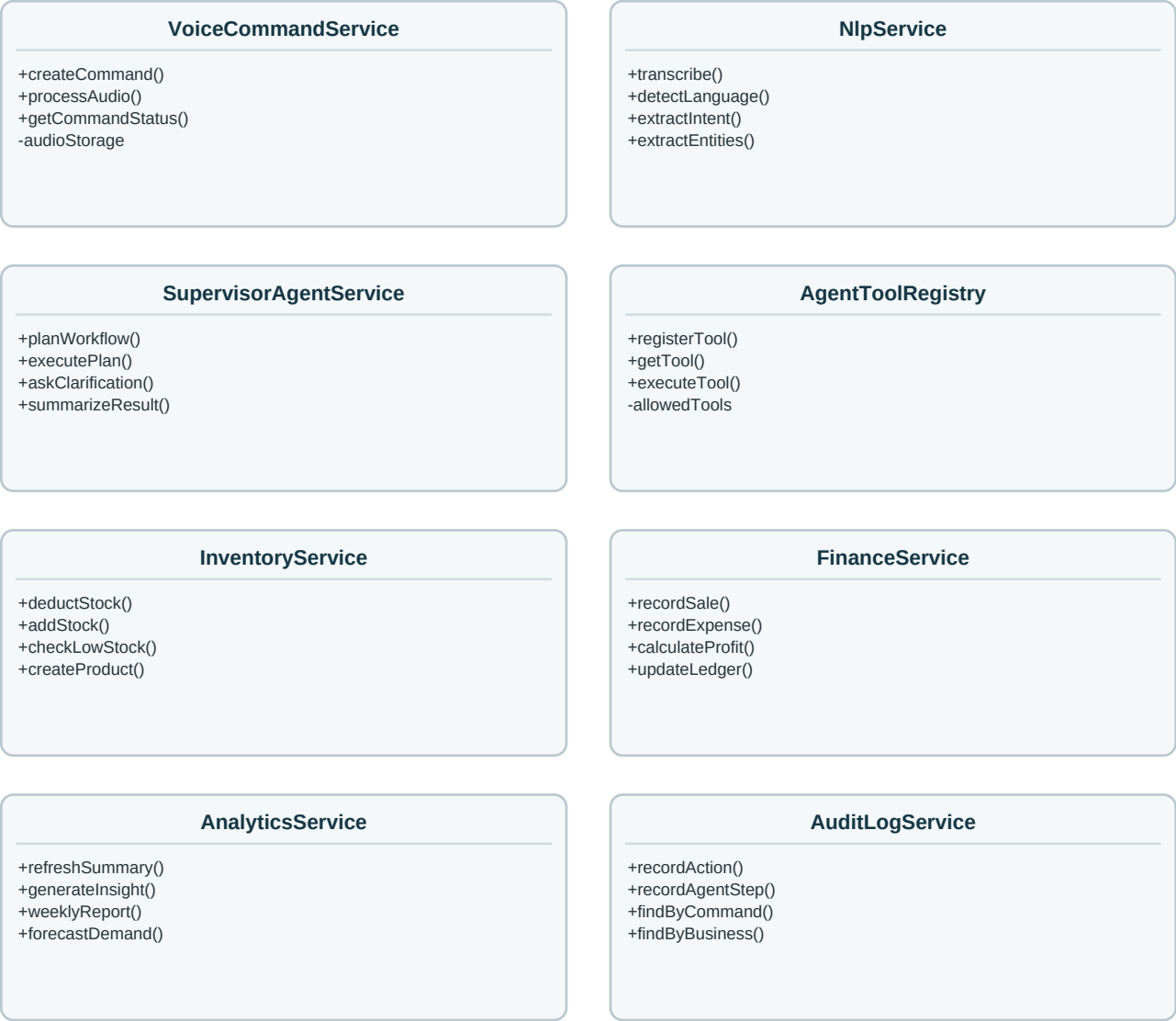
## 5. Layering Inside Each Module

- Controller: receives HTTP requests and maps them to DTOs.
- Service: contains business logic, validation, transactions, and orchestration within the module.
- Repository: performs database access and hides query details.
- DTO: defines request and response shapes.
- Entity/Model: maps application objects to database tables.
- Policy/Validator: handles module-specific rules such as stock checks or payment status checks.

*This keeps the code testable and prevents controllers or agents from directly modifying database tables.*

## 6. Core Class Diagram

Core MVP Classes



7. Detailed Class Responsibilities

| Class                  | Responsibility                                                                        | Important Methods                                               |
|------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| VoiceCommandService    | Creates and tracks a voice command lifecycle from audio/text input to final response. | createCommand, processCommand, markCompleted, markFailed        |
| AudioStorageService    | Stores or discards audio based on privacy settings and retention policy.              | saveAudio, getSignedUrl, deleteAudio                            |
| NlpService             | Converts input into structured business intent and entities.                          | transcribe, normalizeText, detectLanguage, parseCommand         |
| SupervisorAgentService | Plans workflow steps and invokes allowed domain tools.                                | planWorkflow, executeStep, handleToolFailure, buildUserReply    |
| AgentToolRegistry      | Maintains a safe list of callable tools for agents.                                   | registerTool, validateToolCall, executeTool                     |
| InventoryService       | Owns product and stock behavior.                                                      | createProduct, deductStock, addStock, checkThreshold            |
| FinanceService         | Owns sales, expenses, payment status, ledger updates.                                 | recordSale, recordExpense, updatePaymentStatus, calculateProfit |
| AnalyticsService       | Builds dashboard data and insights.                                                   | getSummary, refreshMetrics, generateInsight, buildWeeklyReport  |
| NotificationService    | Creates reminders and alert tasks.                                                    | createReminder, sendLowStockAlert, scheduleNotification         |
| AuditLogService        | Writes immutable user, system, and agent activity logs.                               | recordUserAction, recordAgentAction, recordDataChange           |

8. Key DTOs

```
CreateVoiceCommandDto {
  inputType: 'audio' | 'text'
  text?: string
  audioFileId?: string
  businessId: string
}

ParsedCommandDto {
  commandId: string
  language: string
  intent: 'SALE' | 'EXPENSE' | 'STOCK_UPDATE' | 'REPORT' | 'UNKNOWN'
  entities: CommandEntity[]
  confidence: number
}

WorkflowPlanDto {
  commandId: string
  steps: WorkflowStepDto[]
  requiresConfirmation: boolean
}

RecordSaleDto {
  productId: string
  quantity: number
  unitPrice?: number
  customerId?: string
  paymentStatus: 'PAID' | 'DUE' | 'PARTIAL'
}
```

9. Agent Tool Contracts

Agents should call a restricted set of tools instead of directly writing to the database. Example tools:

| Tool Name                    | Input                                         | Output             | Validation                            |
|------------------------------|-----------------------------------------------|--------------------|---------------------------------------|
| inventory.findProduct        | businessId, productName                       | product candidates | Tenant check, fuzzy match threshold   |
| inventory.deductStock        | productId, quantity, commandId                | new stock level    | Quantity > 0, idempotency, stock rule |
| finance.recordSale           | productId, quantity, unitPrice, paymentStatus | transaction id     | Valid product, price, payment state   |
| analytics.refreshSummary     | businessId, dateRange                         | summary snapshot   | Tenant check                          |
| notifications.createReminder | businessId, reminderType, dueAt               | reminder id        | Valid future date, dedupe             |
| audit.recordAgentStep        | commandId, agentName, action, result          | audit id           | Immutable append-only write           |

## 10. Database Schema Overview

PostgreSQL is the source of truth. Each table should include tenant isolation through `business_id` wherever the record belongs to a business.

| Table             | Purpose                                  | Key Columns                                                                                     |
|-------------------|------------------------------------------|-------------------------------------------------------------------------------------------------|
| users             | Application users.                       | id, name, phone, email, password_hash, preferred_language, created_at                           |
| businesses        | Business profile and settings.           | id, owner_user_id, name, type, location, currency, settings_json                                |
| business_members  | Roles inside a business.                 | id, business_id, user_id, role, status                                                          |
| products          | Product catalog.                         | id, business_id, name, category, unit, cost_price, selling_price, stock_quantity, reorder_level |
| stock_movements   | Inventory movement history.              | id, business_id, product_id, movement_type, quantity, reference_type, reference_id              |
| transactions      | Sales, purchases, payments, adjustments. | id, business_id, type, amount, payment_status, customer_id, occurred_at                         |
| transaction_items | Line items for sales or purchases.       | id, transaction_id, product_id, quantity, unit_price, total                                     |
| expenses          | Business expenses.                       | id, business_id, category, amount, note, occurred_at                                            |
| customers         | Customer data and dues.                  | id, business_id, name, phone, balance_due, last_purchase_at                                     |
| voice_commands    | User command lifecycle.                  | id, business_id, user_id, input_type, transcript, intent, status, confidence                    |
| agent_tasks       | Agent execution steps.                   | id, command_id, agent_name, tool_name, status, input_json, output_json                          |
| insights          | Advisor insights and reports.            | id, business_id, insight_type, message, confidence, created_at                                  |
| reminders         | Follow-ups and alerts.                   | id, business_id, type, message, due_at, status                                                  |
| audit_logs        | Immutable change and decision logs.      | id, business_id, actor_type, actor_id, action, entity_type, entity_id, metadata_json            |

## 11. Important Relationships

- One user can own or belong to many businesses through `business_members`.
- One business has many products, customers, transactions, expenses, reminders, insights, and voice commands.
- One transaction can have many `transaction_items`.
- Each `stock_movement` can reference a transaction, purchase, manual adjustment, or correction.
- Each `voice_command` can produce many `agent_tasks`.
- Audit logs reference any business entity through `entity_type` and `entity_id`.

## 12. SQL Table Definitions

```
CREATE TABLE products (
  id UUID PRIMARY KEY,
  business_id UUID NOT NULL REFERENCES businesses(id),
  name VARCHAR(120) NOT NULL,
  category VARCHAR(80),
  unit VARCHAR(30) NOT NULL DEFAULT 'piece',
  cost_price NUMERIC(12,2) DEFAULT 0,
  selling_price NUMERIC(12,2) DEFAULT 0,
  stock_quantity NUMERIC(12,2) NOT NULL DEFAULT 0,
  reorder_level NUMERIC(12,2) NOT NULL DEFAULT 0,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now(),
  UNIQUE (business_id, name)
);

CREATE TABLE transactions (
  id UUID PRIMARY KEY,
  business_id UUID NOT NULL REFERENCES businesses(id),
  customer_id UUID NULL REFERENCES customers(id),
  type VARCHAR(30) NOT NULL,
  amount NUMERIC(12,2) NOT NULL,
  payment_status VARCHAR(20) NOT NULL,
  source_command_id UUID NULL REFERENCES voice_commands(id),
  occurred_at TIMESTAMP NOT NULL DEFAULT now(),
  created_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE voice_commands (
  id UUID PRIMARY KEY,
  business_id UUID NOT NULL REFERENCES businesses(id),
  user_id UUID NOT NULL REFERENCES users(id),
  input_type VARCHAR(20) NOT NULL,
  transcript TEXT,
  language VARCHAR(20),
  intent VARCHAR(40),
  confidence NUMERIC(5,4),
  status VARCHAR(30) NOT NULL,
  idempotency_key VARCHAR(120) UNIQUE,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  completed_at TIMESTAMP NULL
);
```

## 13. Indexing Strategy

- products: index on business\_id and unique index on business\_id + name.
- transactions: index on business\_id + occurred\_at for reports.
- transactions: index on customer\_id for purchase history and dues.
- voice\_commands: unique idempotency\_key to prevent duplicate processing.
- agent\_tasks: index on command\_id to reconstruct workflow execution.
- audit\_logs: index on business\_id + created\_at and entity\_type + entity\_id.
- stock\_movements: index on product\_id + created\_at for stock history.

## 14. Transaction Handling

For a sale command, inventory and finance updates must be consistent. The sale workflow should run inside a database transaction:

```
recordSaleWorkflow(commandId, saleDto):
  begin transaction
    product = lock product row for update
    validate product belongs to business
    validate quantity > 0
    validate stock rule
    create transaction
    create transaction item
    update product.stock_quantity
    create stock_movement
    create audit_log
```

```
commit transaction
enqueue analytics refresh
enqueue low-stock notification if needed
```

## 15. Main Workflows

| Workflow           | Steps                                                                                                                          |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Sale from voice    | Create command -> NLP parse -> Supervisor plan -> find product -> record sale -> deduct stock -> refresh analytics -> respond. |
| Expense from voice | Create command -> parse amount/category -> Finance records expense -> Analytics refresh -> dashboard update.                   |
| Stock update       | Create command -> parse product/quantity -> Inventory adjusts stock -> Stock movement created -> audit log.                    |
| Low-stock alert    | Inventory checks reorder_level -> Notification creates reminder -> user sees dashboard alert.                                  |
| Weekly report      | Worker fetches analytics -> Advisor generates insight -> report stored -> notification created.                                |
| Clarification      | Low confidence parse -> Supervisor asks question -> user confirms -> original command resumes.                                 |

## 16. Validation Rules

- Every business-owned record must be checked against business\_id.
- Quantity must be positive for sales and purchases.
- Negative stock should either be blocked or require explicit confirmation based on business settings.
- Payment status must be PAID, DUE, or PARTIAL.
- Unit price must be non-negative and defaults to product selling\_price when not spoken.
- Duplicate voice commands must be detected using idempotency\_key and recent command similarity.
- Agent confidence below threshold should trigger clarification instead of automatic write.
- Dangerous actions such as delete, bulk correction, or customer-facing invoice sending require user confirmation.

## 17. Error Handling

| Error Case            | Handling                                                                                |
|-----------------------|-----------------------------------------------------------------------------------------|
| Product not found     | Return candidates or ask user to create/select product.                                 |
| Low speech confidence | Ask for confirmation or request repeat input.                                           |
| Insufficient stock    | Warn user, ask whether to allow negative stock if policy permits.                       |
| Duplicate command     | Return existing command result instead of creating a second transaction.                |
| Agent tool failure    | Mark agent_task failed, retry if safe, otherwise ask user or escalate to manual review. |
| Database failure      | Rollback transaction and keep command status as FAILED or RETRY_PENDING.                |
| Analytics failure     | Do not fail sale; enqueue retry because analytics is eventually consistent.             |

## 18. API Endpoint Map

| Endpoint                    | Method   | Purpose                                |
|-----------------------------|----------|----------------------------------------|
| /auth/login                 | POST     | Login or OTP verification.             |
| /businesses                 | GET/POST | List or create businesses.             |
| /voice/commands             | POST     | Create voice/text command.             |
| /voice/commands/:id         | GET      | Fetch command status and result.       |
| /agent/commands/:id/confirm | POST     | Confirm clarification or risky action. |
| /products                   | GET/POST | List or create products.               |
| /products/:id/stock         | PATCH    | Manual stock adjustment.               |
| /transactions               | GET/POST | List or create transactions.           |
| /expenses                   | GET/POST | List or create expenses.               |
| /analytics/dashboard        | GET      | Dashboard summary.                     |
| /insights                   | GET      | Advisor insights.                      |
| /reminders                  | GET/POST | List or create reminders.              |

## 19. Configuration and Environment

```
DATABASE_URL=postgresql://...
REDIS_URL=redis://...
JWT_SECRET=...
OBJECT_STORAGE_BUCKET=vyaparflow-files
SPEECH_PROVIDER=openai|google|azure
LLM_PROVIDER=openai|local|other
COMMAND_CONFIDENCE_THRESHOLD=0.75
ALLOW_NEGATIVE_STOCK=false
AUDIO_RETENTION_DAYS=0
QUEUE_NAME=vyaparflow-jobs
```

## 20. Testing Plan at LLD Level

- Unit tests for InventoryService, FinanceService, NlpService parsing, and validation utilities.
- Repository tests for product lookup, transaction creation, and stock movement history.
- Integration tests for sale workflow, expense workflow, low-stock notification, and duplicate command handling.
- Agent tool tests to ensure only registered tools can be called.
- Database transaction tests to verify rollback when stock or finance updates fail.
- E2E tests for voice command creation through dashboard update.

## 21. Suggested Implementation Order

- Set up repository, backend app, database connection, migrations, and shared DTO package.
- Implement Auth, Businesses, Users, and tenant context middleware.
- Implement Products, Inventory, Finance, Transactions, and Stock Movements.
- Implement VoiceCommand lifecycle with text input first, then add audio upload.
- Implement NLP parser with intent/entity extraction.
- Implement SupervisorAgentService and restricted AgentToolRegistry.
- Connect sale, expense, and stock update workflows end to end.
- Add Analytics dashboard, low-stock reminders, audit logs, and report generation.
- Add tests, seed data, observability, and production deployment configuration.

## 22. Closing LLD Summary

### Presenter:

To summarize, this LLD defines VyaparFlow as a modular backend with clear boundaries. Voice and NLP modules understand user commands. The Supervisor Agent plans the workflow. Domain services such as Inventory and Finance perform validated business operations. PostgreSQL stores the source of truth, Redis supports temporary state, and audit logs preserve traceability.

### Presenter:

The most important low-level design decision is to keep AI agents away from direct database writes. Agents should call controlled tools, and those tools should delegate to domain services that validate input, enforce tenant isolation, use transactions, and create audit logs.

*LLD takeaway: use modular code, deterministic domain services, restricted agent tools, transactional database updates, and auditability for every business-critical action.*

Recommended next step: create detailed API contracts, sequence diagrams, and ER diagrams for the selected implementation stack.